Department of Artificial Intelligence and Machine Learning

Course Code: AI372IA **Course:** Stream Processing and Analytics

Environment, Tools and Configurations

Project Title	Smart Agriculture Crop Health Monitoring
Student Name/s	Aanish Khan Daksh Chauhan Pranav Pratyush
Student USN/s	1RV23AI002 1RV23AI028 1RV23AI133
Faculty Guide/s	Dr. Vijayalakshmi M N (Associate Professor) Dr. S. Anupama Kumar (Associate Professor)

1. Development and Runtime Environment

AgriKafka-Agent is planned as a locally deployable, containerized streaming prototype. Docker Compose will coordinate the broker, stream processor, storage and application services. The configuration below is a proposed reproducible baseline; component versions, resource limits and final port mappings should be pinned in the project repository before the final demonstration.

Item	Proposed configuration
Operating system	Windows 10/11 (64-bit) with WSL 2, or a compatible Linux environment
Container platform	Docker Desktop with Docker Compose; use containers to keep service dependencies reproducible
Hardware	Suggested baseline: 8 GB RAM minimum, 16 GB recommended, 4 CPU cores and approximately 6 GB free disk space; actual needs depend on enabled services
Language / runtime	Python for ingestion, data-quality checks, API/advisory services and supporting scripts; use a pinned Python container image
Stream processing	Apache Kafka for event transport and PySpark Structured Streaming for event-time processing and risk detection
Development tools	Visual Studio Code, Git, PowerShell or a Linux shell, Docker CLI and a modern browser
Network	Internet required for initial image downloads and public weather/environmental API access; cached historical data should support offline development tests

2.Tools and Technologies

Layer	Tool / technology	Purpose in AgriKafka-Agent
Message broker	Apache Kafka (version to pin)	Buffers and distributes environmental observations and risk events between services
Stream processing	PySpark Structured Streaming (version to pin)	Event-time processing, temporal features, contextual risk detection and stream-level metrics
Ingestion / utilities	Python + HTTP client (versions to pin)	Fetches public API data, normalizes records, validates schema and publishes to Kafka
Storage	PostgreSQL (version to pin)	Persists processed observations, alerts, evaluation records and metadata
AI advisory	AI model/API or local model (implementation decision pending)	Explains selected alerts using evidence and approved guidance; must communicate uncertainty
Dashboard / API	Web dashboard + backend (framework to select)	Displays field/location risk, supporting evidence, data freshness, latency and system health
Containerization	Docker + Docker Compose	Reproducible service setup and coordinated startup/shutdown
Testing	pytest + scripted fault injection	Tests transformations, detection logic, malformed/late data handling and recovery behaviour
Data sources	Public weather/environmental APIs; historical dataset; optional satellite-derived indicators	Supplies environmental context; modelled data must not be represented as physical sensor ground truth

3. System Configuration

3.1 Proposed Docker services

Service	Role	Port / exposure
kafka	Message broker for raw observations and downstream events	Internal broker listener; host mapping to be configured
ingestion	Polls available APIs or replays historical records and publishes normalized events	No host port required by default
spark-processor	Consumes Kafka events, computes temporal/contextual features and emits risk events	No host port required by default
postgres	Stores processed observations, alerts and evaluation metadata	Optional local DB port; restrict to localhost
advisor	Creates evidence-grounded explanations for selected alerts	Internal service by default
dashboard	User interface for risk alerts, trends and system health	Proposed localhost port: 8000
kafka-ui (optional)	Development-only view of topics, partitions and consumer groups	Proposed localhost port: 8085

3.2 Suggested Kafka topics

Topic	Content	Purpose
agri.raw	Normalized environmental observations with location and event timestamp	Input stream
agri.validated	Validated records and quality flags	Separates data checks from risk analysis
agri.risk-events	Risk score, contributing evidence and detector metadata	Alerts and advisor input
agri.dlq	Malformed or unprocessable records with error metadata	Inspection and replay
agri.ops-metrics	Processing lag, throughput, errors and recovery events	Observability and evaluation

3.3 Key configuration principles

- **Secrets:** Store API keys and credentials in an untracked .env file; provide a .env.example with placeholder values and never commit secrets.
- **Reproducibility:** Pin compatible versions for Kafka, Spark, Python libraries and database images in dependency and Compose files before benchmarking.
- **Event-time handling:** Keep source timestamps and ingestion timestamps; document the chosen window, watermark and late-event policy after examining the actual data cadence.
- **Fault tolerance:** Configure Spark checkpoints on a persistent volume and use safe Kafka producer/consumer settings. Test recovery and duplicate handling instead of assuming exactly-once end-to-end behaviour.
- **Data provenance:** Label public/modelled observations, historical records, and injected fault scenarios separately. Do not label modelled soil moisture as physical sensor measurements.
- **Security and access:** Expose dashboards only on localhost for development; keep database and broker ports internal unless external access is required.

4 Setup and Execution

1. Install Docker Desktop and enable WSL 2 on Windows, or install Docker Engine and the Compose plugin on Linux. Wait until Docker reports that its engine is running.
2. Clone or unzip the project repository, open its root directory in PowerShell or VS Code, and review the README and Compose configuration.
3. Create the local environment file from the supplied template (for example, copy **.env.example** to **.env**). Add required API credentials only if the selected data source requires them.
4. Build and start the configured services with **docker compose up --build**. The exact command may be wrapped in a project start script once the repository is finalized.
5. Check service status with **docker compose ps** and inspect startup errors with **docker compose logs -f**. Confirm that Kafka, the processor and PostgreSQL become healthy.
6. Open the dashboard at the configured localhost URL (proposed: **http://localhost:8000**). If enabled, open Kafka UI at **http://localhost:8085**.
7. Run the automated tests and controlled fault scenarios; record detector metrics, processing latency, throughput, and recovery observations before reporting results.

Useful commands: `docker compose up --build` (start), `docker compose ps` (status), `docker compose logs -f` (logs), `docker compose down` (stop). Use `docker compose down -v` only when intentionally deleting persisted database/checkpoint data.

Configuration status: This document describes a proposed setup for the AgriKafka-Agent prototype. Exact component versions, topic partition counts, resource limits, API endpoints, final dashboard framework and repository-specific scripts must be confirmed during implementation; no completed deployment or benchmark results are implied.